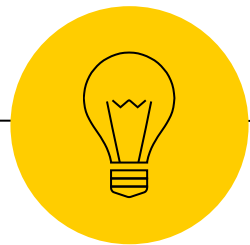




University of Camerino
School of Sciences and Technology
Master's Degree in Computer Science (LM-18)

Automated data pipeline for LLM models usage and cost analysis



A.A. 2025/2026

Professor
Prof. Massimo Callisto
De Donato

Students
Bonsignore Christian
Perini Michele

● Contents of the presentation

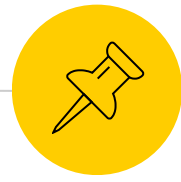
1 Project Description

2 Methodology &
Technologies

3 Implementation

4 Results

5 Future Improvements



The Problem

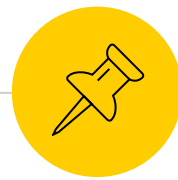
Data fragmentation

Raw conversation metadata
separation from granular
call-level token usage
metrics.

Unpredictability

Unpredictable cost spikes
and dynamic latencies
across heterogeneous LLM
endpoints.

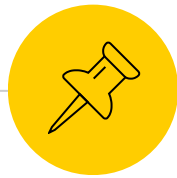
The goal would be an idempotent, self-healing pipeline for metric aggregation, cost estimation, and statistical correlation.



The Idea

Building an automated local data warehouse that handles intermittent API schemas, missing cost parameters, and updates without double-counting usage deltas.

This allow us to systematically track, model, and visualize the relationships between prompt design, model throughput, latency, and operational costs.



The Architecture

Container

Infrastructure:

Docker Compose
orchestrating
microservices.

Storage Layer:

PostgreSQL 15 (relational
schema, indexed tables,
and materialized views).

Ingestion & ETL:

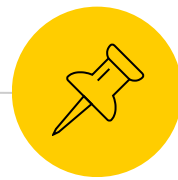
Python 3.11 with
SQLAlchemy 2.0 and
HTTP retry handlers.

Orchestration:

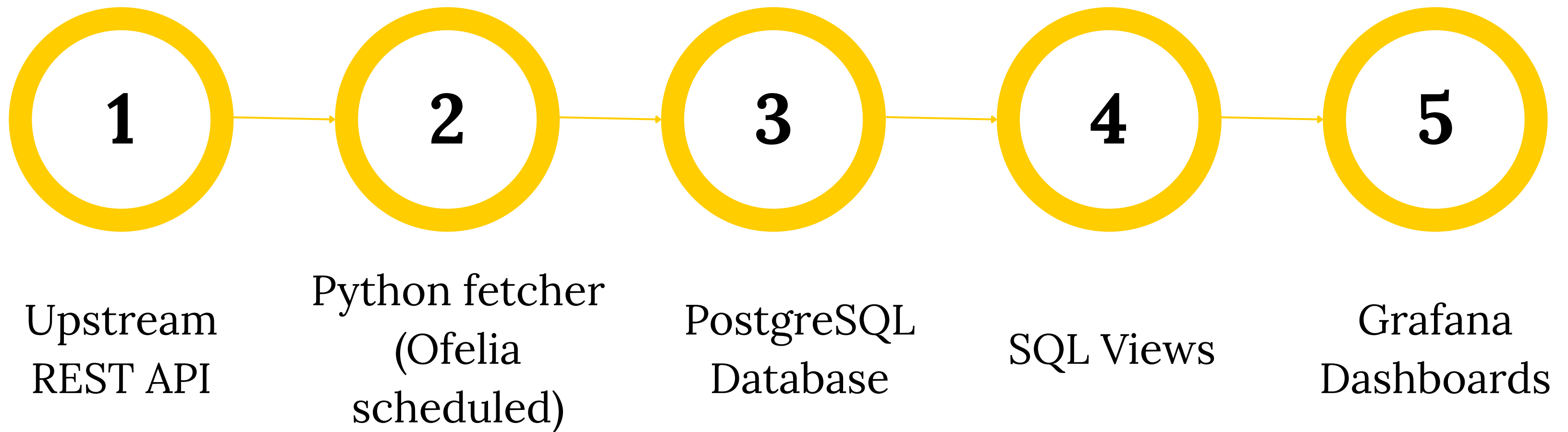
Ofelia (Docker-native
cron daemon running
sync every 30 mins).

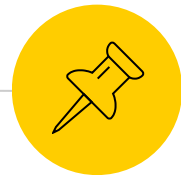
Visualization:

Grafana (auto-
provisioned dashboard
with persistency).



The Flow





The Strategy

Two-Phase Fetching:

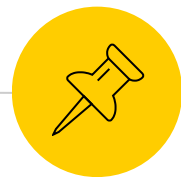
Sequential pull of */conversations* followed by differential */token-usage* fetches.

Snapshot Tracking:

Uses *updated_at* timestamps to bypass unnecessary API calls on unchanged threads.

Idempotent Increments:

Incremental token history stored using unique source constraints (*conv_id*, *source_updated_at*) to prevent double-counting.

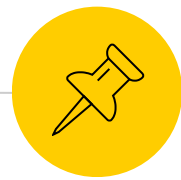


The Cost Estimation

1

The pipeline first checks for an explicit *cost_usd* value with the upstream API. If it's missing, the system falls back to estimating the expense locally.





The Cost Estimation

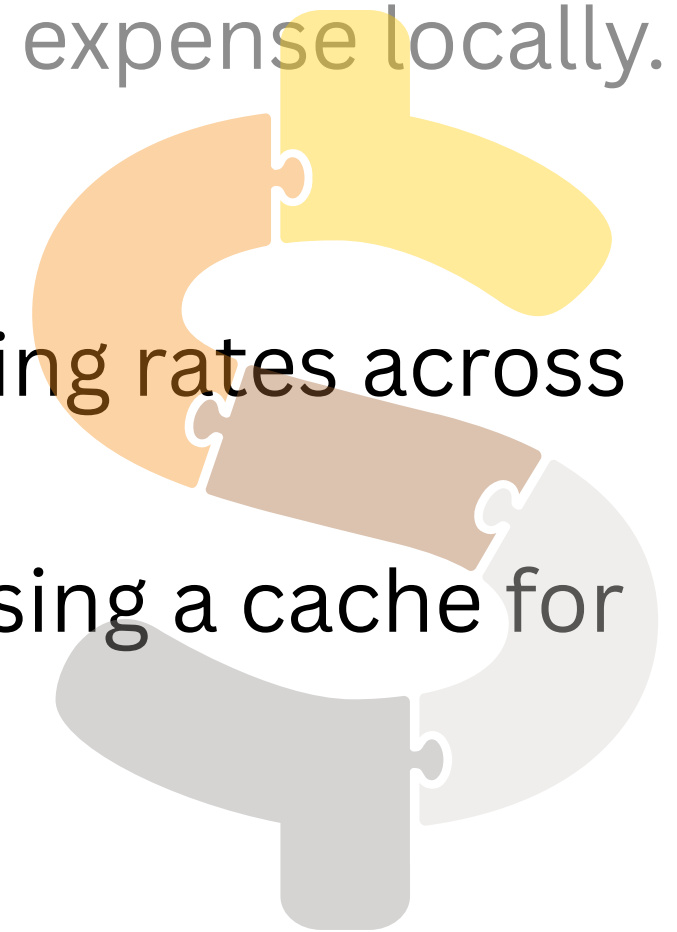
1

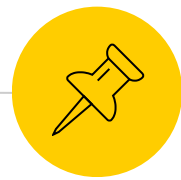
The pipeline first checks for an explicit *cost_usd* value with the upstream API. If it's missing, the system falls back to estimating the expense locally.

Local estimations rely on:

2

- a configuration file (*model_pricing.json*) that stores pricing rates across multiple categories;
- a script *pricing.py* that loads these rates into memory using a cache for performance.





The Cost Estimation

1

The pipeline first checks for an explicit *cost_usd* value with the upstream API. If it's missing, the system falls back to estimating the expense locally.

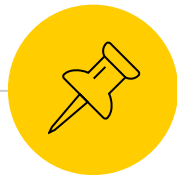
Local estimations rely on:

2

- a configuration file (*model_pricing.json*) that stores pricing rates across multiple categories;
- a script *pricing.py* that loads these rates into memory using a cache for performance.

3

The engine's matching algorithm handles both exact model names and version-suffixed variants. If the API returns a dynamically dated model like *gpt-4.1-2025-04-14*, the engine maps it to the correct base pricing tier.



The Database

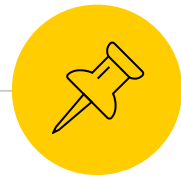
Tables: *api_sessions*, *llm_calls*, *token_usage_history*

Index Strategy: Compound indexes on *conv_id*, *updated_at*, *model*, and time parameters.

The views:

conversation_metrics: Calculates output/input token ratios, average call latencies, and normalized cost per 1,000 tokens

model_call_metrics: Aggregates usage metrics, total token counts, and cost distributions per model



The Dashboard

Real-time KPIs:

Total conversations, call counts, cumulative tokens, cost, and overall output/input ratios.

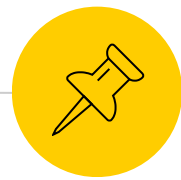
Latency & Cost Profiling:

Breakdown of cost distribution by model and call-type latency distributions.

Outlier Detection:

Automated table isolating top resource-draining sessions sorted by cost per 1,000 tokens.





The Analysis

Scatter Plots:

Visualizing input vs. output tokens and token volume vs. aggregate cost.

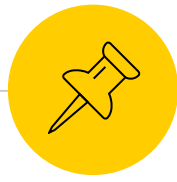
Pearson

Correlation Matrix:

Quantitative tracking of dependencies among duration, calls, costs, and token lengths.

Demo Data Generation:

Built-in seed engine creating realistic statistical correlations for local verification.



The Future Directions

Streaming & Event-Driven Architecture:

Replace periodic polling (Ofelia) with a message broker (e.g., Apache Kafka / RabbitMQ) for sub-second live ingestion.

Multi-Tenant Isolation:

Expand table schemas to enforce strict organization-level Row-Level Security (RLS) in PostgreSQL.

Anomalous Behavior Detection:

Implement real-time sliding-window alerting for out-of-budget token consumption or abnormal model latency spikes.

Dynamic Price Invalidation:

Automatically fetch active pricing catalogs directly from provider endpoints instead of relying on static local JSON configurations.



**Thank you for
your attention**